Can we be sure that G will be the closest point to P, though? That would have been ideal, but unfortunately that's not the case. Figure 9.20 shows this first step in the algorithm, traversing a path in the tree from the root to a leaf, to find the smallest region containing P.

Listing 9.9 The nearestNeighbor method

```
function nearestNeighbor(node, target, (nnDist, nn)=(inf,
null))              #1
  if node == null
then                                                        #2
    return (nnDist, nn)

else
              #3
    dist ← distance(node.point, target)
        #4
    if dist < nnDist then
        #5
      (nnDist, nn) ← (dist, node.point)
    if compare(target, node) < 0
then                                          #6
      closeBranch ← node.left
      farBranch ← node.right
    else
                                                          #7
      closeBranch ← node.right
      farBranch ← node.left
    (nnDist, nn) ← nearestNeighbor(closeBranch, target,
(nnDist, nn))       #8
    if splitDistance(target, node) < nnDist
then                                #9
      (nnDist, nn) ← nearestNeighbor(farBranch, target,
(nnDist, nn))        #10
    return (nnDist,
nn)                                                       #11
```